CoPL
Object Oriented Programming

Crashcourse

Daan Wichmann

Radboud University

10 03 2026

Before we start

- Lot's of topics to cover, limited time per topic
- Assuming basic Python knowledge from Programming 1 & Programming 2.

Objects & Methods

What are objects?

What are objects?

Objects

Objects are Python's abstraction for data. All data in a Python program is represented by objects or by relations between objects.

What are objects?

What are objects?

The following are objects in Python:

What are objects?

The following are objects in Python:

- Integers (e.g. 127)

What are objects?

The following are objects in Python:

- Integers (e.g. 127)
- Strings (e.g. "CognAC")

What are objects?

The following are objects in Python:

- Integers (e.g. 127)
- Strings (e.g. "CognAC")
- Lists (e.g. ["Damai", "Daan", "Ella"])

What are objects?

The following are objects in Python:

- Integers (e.g. 127)
- Strings (e.g. "CognAC")
- Lists (e.g. ["Damai", "Daan", "Ella"])
- **Every value in Python!**

What are methods?

What are methods?

Methods

Methods are functions that belong to a certain **type of object**.

What are methods?

Methods

Methods are functions that belong to a certain **type of object**.

- You call methods on an object.

What are methods?

Methods

Methods are functions that belong to a certain **type of object**.

- You call methods on an object.
- Methods that belong to a certain type of object, can not be called on another type of object

What are methods?

You have already seen a lot of methods!

What are methods?

You have already seen a lot of methods!

```
my_list = [1, 2, 3]
```

What are methods?

You have already seen a lot of methods!

```
my_list = [1, 2, 3]
my_list.append(4)
```

What are methods?

You have already seen a lot of methods!

```
my_list = [1, 2, 3]
my_list.append(4)
print(my_list)
>>> [1, 2, 3, 4]
```

What are methods?

You have already seen a lot of methods!

```
my_list = [1, 2, 3]
my_list.append(4)
print(my_list)
>>> [1, 2, 3, 4]
```

append is a **method** of lists.

What are methods? Another example.

What are methods? Another example.

```
my_string = "daan"  # Note: all lowercase
```

What are methods? Another example.

```
my_string = "daan"  # Note: all lowercase
my_capitalized_string = my_string.capitalize()
```

What are methods? Another example.

```
my_string = "daan"  # Note: all lowercase
my_capitalized_string = my_string.capitalize()
print(my_capitalized_string)
>>> "Daan"
```

What are methods? Another example.

```
my_string = "daan"  # Note: all lowercase
my_capitalized_string = my_string.capitalize()
print(my_capitalized_string)
>>> "Daan"
```

capitalize is a **method** of strings.

What are methods?

What are methods?

```
my_list = [1, 2, 3]
```

What are methods?

```
my_list = [1, 2, 3]
my_list.capitalize()
```

What are methods?

```
my_list = [1, 2, 3]
my_list.capitalize()
>>> AttributeError
```

What are methods?

```
my_list = [1, 2, 3]
my_list.capitalize()
>>> AttributeError
```

Why? *'capitalize' is not a method of 'list'*

Classes

What are classes?

What are classes?

Class (definition)

A class is a blueprint for a type of object, defining it's **data attributes** and **methods**.

What are classes?

Class (definition)

A class is a blueprint for a type of object, defining it's **data attributes** and **methods**.

Terminology

We call an object from a certain class, an **instance** of that class. E.g. "OOP" is an instance of the class str (string).

Why do we use classes?

Why do we use classes?

Why do we use classes?

- Python only gives us a limited amount of types, integers, floats, strings, etcetera.

Why do we use classes?

- Python only gives us a limited amount of types, integers, floats, strings, etcetera.
- With classes we can extend this and model anything we want!

Why do we use classes?

- Python only gives us a limited amount of types, integers, floats, strings, etcetera.
- With classes we can extend this and model anything we want!
  - Like students, bank accounts, courses, shopping cart.

Why do we use classes?

- Python only gives us a limited amount of types, integers, floats, strings, etcetera.
- With classes we can extend this and model anything we want!
  - Like students, bank accounts, courses, shopping cart.
- Encapsulation (later)

Why do we use classes?

- Python only gives us a limited amount of types, integers, floats, strings, etcetera.
- With classes we can extend this and model anything we want!
  - Like students, bank accounts, courses, shopping cart.
- Encapsulation (later)
- Seperation of Concerns (later)

What do classes consist of?

What do classes consist of?

- Methods (behaviour)

What do classes consist of?

- Methods (behaviour)
- Data Attributes (properties/fields)

What do classes consist of?

What do classes consist of?

Data Attributes:

What do classes consist of?

Data Attributes:

- Variables that belong to a single object

What do classes consist of?

Data Attributes:

- Variables that belong to a single object
- For example a person can have the following data variables (i.e. data attributes) which we would like to store:

What do classes consist of?

Data Attributes:

- Variables that belong to a single object
- For example a person can have the following data variables (i.e. data attributes) which we would like to store:
  - first name (string)
  - last name (string)
  - age (int)

What do classes consist of?

Data Attributes:

- Variables that belong to a single object
- For example a person can have the following data variables (i.e. data attributes) which we would like to store:
  - first name (string)
  - last name (string)
  - age (int)
- These variables can be of any type! (Strings, integers, lists, and also objects of other classes)

Methods

- You have already seen a lot of methods

Methods

- You have already seen a lot of methods
- For example, for a person we could have:

Methods

- You have already seen a lot of methods
- For example, for a person we could have:
  - Speak
  - Dance
  - Study

Methods

- You have already seen a lot of methods
- For example, for a person we could have:
  - Speak
  - Dance
  - Study
- Every class needs one special method:

Methods

- You have already seen a lot of methods
- For example, for a person we could have:
  - Speak
  - Dance
  - Study
- Every class needs one special method:
  - **The constructor**

What is a constructor?

What is a constructor?

Constructor

In Python, a **constructor** is a special method called when an object is created. It initializes all data attributes.

The constructor

To create objects of a certain class:

The constructor

To create objects of a certain class:

```
my_string = "CognAC"  # Class str
my_int = 18  # Class int
my_list = [18, 2003]  # Class list
```

The constructor

To create objects of a certain class:

```
my_string = "CognAC"  # Class str
my_int = 18  # Class int
my_list = [18, 2003]  # Class list
```

But what if we have a new type of object of a new class, for example a Fraction class

The constructor

To create objects of a certain class:

```
my_string = "CognAC"  # Class str
my_int = 18  # Class int
my_list = [18, 2003]  # Class list
```

But what if we have a new type of object of a new class, for example a Fraction class *We use the constructor!*

The constructor

```
# We import the class from the module 'fractions'
# Classes always start with a capital lette
from fractions import Fraction
```

The constructor

```
# We import the class from the module 'fractions'
# Classes always start with a capital lette
from fractions import Fraction
# We call the constructor by 'calling the class',
# this creates a new object for us!
# The constructor takes two arguments:
# numerator and denominator (in this case 8 and 10)
my_fraction = Fraction(8, 10)  # Fraction of 8/10th
```

Making our own class!

Making our own class!

- We now have all ingredients to create our own classes!

Making our own class!

- We now have all ingredients to create our own classes!
- Let's say we want to model a person in python

Making our own class!

- We now have all ingredients to create our own classes!
- Let's say we want to model a person in python
- We want it to have a *first name*, *last name*, *age* (data attributes)

Making our own class!

- We now have all ingredients to create our own classes!
- Let's say we want to model a person in python
- We want it to have a *first name*, *last name*, *age* (data attributes)
- And we want a person to 'speak' i.e. print some text (methods)

Making our own class!

- We now have all ingredients to create our own classes!
- Let's say we want to model a person in python
- We want it to have a *first name*, *last name*, *age* (data attributes)
- And we want a person to 'speak' i.e. print some text (methods)
- We create our own class **Person**

Creating our Person class

```
class Person():
```

Creating our Person class

```
class Person():

    # The constructor
    def __init__(self, first_name, last_name, age):
        self.first_name = first_name  # Data attribute
        self.last_name = last_name  # Data attribute
        self.age = age  # Data attribute
```

Creating our Person class

```python
class Person():

    # The constructor
    def __init__(self, first_name, last_name, age):
        self.first_name = first_name  # Data attribute
        self.last_name = last_name  # Data attribute
        self.age = age  # Data attribute

    # Method
    def speak(self):
        print(f"Hey, my name is {self.first_name}")
```

The 'self' parameter

The 'self' parameter

- The first parameter in a constructor definition is always named **self**.

The 'self' parameter

- The first parameter in a constructor definition is always named **self**.
- This refers to the object itself, and is necessary for declaring any data attributes attached to the object.

The 'self' parameter

- The first parameter in a constructor definition is always named **self**.
- This refers to the object itself, and is necessary for declaring any data attributes attached to the object.
- It is automatically filled in by the interpreter with the object it is called on.

Creating our Person class

```python
class Person():

    # The constructor
    def __init__(self, first_name, last_name, age):
        self.first_name = first_name  # Data attribute
        self.last_name = last_name  # Data attribute
        self.age = age  # Data attribute

    # Method
    def speak(self):
        print(f"Hey, my name is {self.first_name}")
```

Data Attributes

Accessing a data attribute:

Data Attributes

Accessing a data attribute:

```
person = Person("Daan", "Wichmann", "Secret")
```

Data Attributes

Accessing a data attribute:

```
person = Person("Daan", "Wichmann", "Secret")
print(person.first_name)
>>> "Daan"
```

Data Attributes

Re-assigning an attribute:

Data Attributes

Re-assigning an attribute:

```
person = Person("Daan", "Wichmann", "Secret")
```

Data Attributes

Re-assigning an attribute:

```
person = Person("Daan", "Wichmann", "Secret")
person.first_name = "Damai"
print(person.first_name)
>>> "Damai"
```

Our beautiful class

```python
class Person():

    # The constructor
    def __init__(self, first_name, last_name, age):
        self.first_name = first_name  # Attribute
        self.last_name = last_name  # Attribute
        self.age = age  # Attribute

    # Method
    def speak(self):
        print(f"Hey, my name is {self.first_name}")
```

Calling our method

Calling our method

```
person = Person("Daan", "Wichmann", "Secret")
```

Calling our method

```
person = Person("Daan", "Wichmann", "Secret")
person.speak()
>>> Hey, my name is Daan
# Note that we don't fulfill the 'self'
# parameter, but the interpreter
# does it automatically for us.
```

Why?

Why do we want to use classes?

- If we want to to represent something that is not already in Python
- To make collaboration easier
- If we want to 'group together' functionality

Objects & References

What do our variables store exactly?

References

```
person1 = Person("Daan", "Wichmann", "Secret")
```

References

```
person1 = Person("Daan", "Wichmann", "Secret")
person2 = person1
```

References

```
person1 = Person("Daan", "Wichmann", "Secret")
person2 = person1
person3 = Person("Damai", "Jiworo", "Secret")
```

Memory model

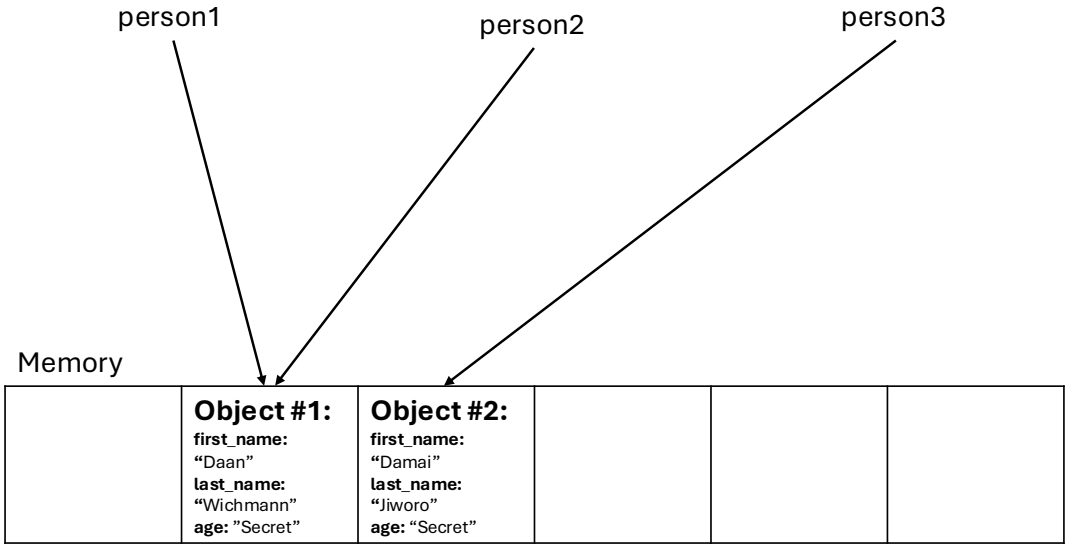

Figure 1: Simplified memory model

Small quiz

```
person1 = Person("Damai", "Jiworo", "Secret")
```

Small quiz

```
person1 = Person("Damai", "Jiworo", "Secret")
person2 = person1
```

Small quiz

```
person1 = Person("Damai", "Jiworo", "Secret")
person2 = person1
person3 = Person("Daan", "Wichmann", "Secret")
```

Small quiz

```
person1 = Person("Damai", "Jiworo", "Secret")
person2 = person1
person3 = Person("Daan", "Wichmann", "Secret")
person2.first_name = "Emma"
```

Small quiz

```
person1 = Person("Damai", "Jiworo", "Secret")
person2 = person1
person3 = Person("Daan", "Wichmann", "Secret")
person2.first_name = "Emma"
print(person1.first_name)  # What is the output?
```

Small quiz

```
person1 = Person("Damai", "Jiworo", "Secret")
person2 = person1
person3 = Person("Daan", "Wichmann", "Secret")
person2.first_name = "Emma"
print(person1.first_name)  # What is the output?
>>> "Emma"
```

Another one

```
person1 = Person("Damai", "Jiworo", "Secret")
```

Another one

```
person1 = Person("Damai", "Jiworo", "Secret")
person2 = Person("Damai", "Jiworo", "Secret")
```

Another one

```
person1 = Person("Damai", "Jiworo", "Secret")
person2 = Person("Damai", "Jiworo", "Secret")
person2.first_name = "Emma"
```

Another one

```
person1 = Person("Damai", "Jiworo", "Secret")
person2 = Person("Damai", "Jiworo", "Secret")
person2.first_name = "Emma"
print(person1.first_name)  # What is the output?
```

Another one

```
person1 = Person("Damai", "Jiworo", "Secret")
person2 = Person("Damai", "Jiworo", "Secret")
person2.first_name = "Emma"
print(person1.first_name)  # What is the output?
>>> "Damai"
```

Part 1 of 3 done! Break?

Inheritance

New idea... new problem...

New idea... new problem...

- We have now modeled a person with our *Person* class

New idea... new problem...

- We have now modeled a person with our *Person* class
- What if we want to model a student?

New idea... new problem...

- We have now modeled a person with our *Person* class
- What if we want to model a student?
- A student is also a person!

New idea... new problem...

- We have now modeled a person with our *Person* class
- What if we want to model a student?
- A student is also a person!
  - **Duplicate code**

New idea... new problem...

- We have now modeled a person with our *Person* class
- What if we want to model a student?
- A student is also a person!
  - **Duplicate code**
- What is the best way to model this?

New idea... new problem...

- We have now modeled a person with our *Person* class
- What if we want to model a student?
- A student is also a person!
  - **Duplicate code**
- What is the best way to model this?
  - using **Inheritance!**

So, what is inheritance?

Inheritance

Lets look at the definition in the context of biology

Inheritance

Lets look at the definition in the context of biology

Inheritance (Biology)

Inheritance is the way that genetic information is passed from a parent to a child. Members of the same biological family tend to have similar characteristics

Lets translate this to OOP!

Lets translate this to OOP!

Inheritance (OOP)

Inheritance is the concept in which a class can *inherit* **data attributes** and **methods** from another class. Here, the class that inherits, is called the **derived class**. The class that the derived class inherits from, is called the **base class**.

Lets translate this to OOP!

Inheritance (OOP)

Inheritance is the concept in which a class can *inherit* **data attributes** and **methods** from another class. Here, the class that inherits, is called the **derived class**. The class that the derived class inherits from, is called the **base class**.

Alternative terminology

Inheriting is sometimes called **deriving**. So you can also say, the derived class **derives** data attributes and methods from it's base class.

In the case of our example:

Inheritance (OOP)

Inheritance is the concept in which a class (**Student**) can *inherit* data attributes (**first_name, last_name, age**) and methods (**speak()**) from another class (**Person**). The class that the derived class inherits from, is called the **base class**.

Alternative terminology

Inheriting is sometimes called **deriving**. So you can also say, the derived class **derives** data attributes and methods from it's base class.

What can we do with inheritance?

- Create a **hierarchy** of classes (i.e. a class hierarchy :D) which describes their relationship (e.g. a Student is a Person, but not the other way around)
- Avoid writing duplicate code!
- **Encapsulation** (more on that later)

Let’s look at the class hierarchy of our example!

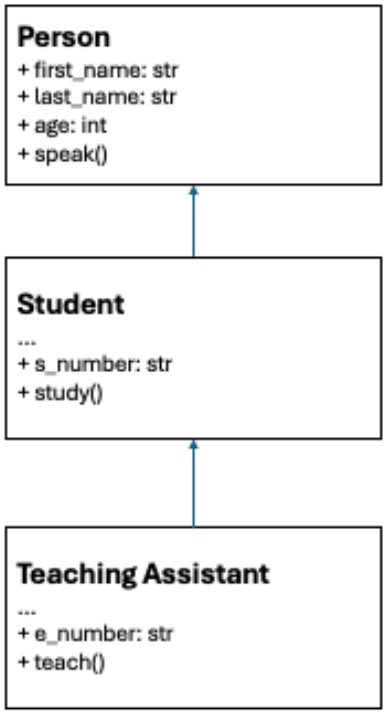

Figure 2: our hierarchy

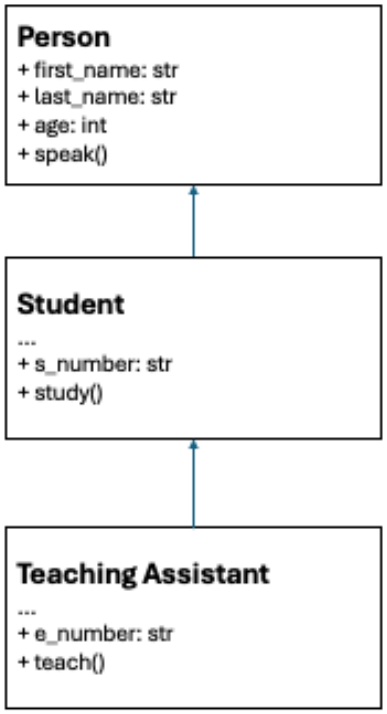

Figure 2: our hierarchy

Note: Inheriting classes (Student) can also be inherited from (TeachingAssistant)!

Surprise quiz (no grade)

Inheritance quiz

What is the *base class* of *Student*?

Inheritance quiz

What is the *base class* of *Student*?

- **Answer:** Person

Inheritance quiz

What is the *base class* of *Student*?

- **Answer:** Person

Which class is *Student* a *derived class* of?

Inheritance quiz

What is the *base class* of *Student*?

- **Answer:** Person

Which class is *Student* a *derived class* of?

- **Answer:** Person

Inheritance quiz

What is the *base class* of *Student*?

- **Answer:** Person

Which class is *Student* a *derived class* of?

- **Answer:** Person

What is a *derived class* of *Student*?

Inheritance quiz

What is the *base class* of *Student*?

- **Answer:** Person

Which class is *Student* a *derived class* of?

- **Answer:** Person

What is a *derived class* of *Student*?

- **Answer:** TeachingAssistant

Inheritance quiz

What is the *base class* of *Student*?

- **Answer:** Person

Which class is *Student* a *derived class* of?

- **Answer:** Person

What is a *derived class* of *Student*?

- **Answer:** TeachingAssistant

Which data attributes does *TeachingAssistant* inherit?

Inheritance quiz

What is the *base class* of *Student*?

- **Answer:** Person

Which class is *Student* a *derived class* of?

- **Answer:** Person

What is a *derived class* of *Student*?

- **Answer:** TeachingAssistant

Which data attributes does *TeachingAssistant* inherit?

- **Answer:** first_name, last_name, age, and s_number

Inheritance quiz

What is the *base class* of *Student*?

- **Answer:** Person

Which class is *Student* a *derived class* of?

- **Answer:** Person

What is a *derived class* of *Student*?

- **Answer:** TeachingAssistant

Which data attributes does *TeachingAssistant* inherit?

- **Answer:** first_name, last_name, age, and s_number

Which class is *TeachingAssistant* a *derived class* of?

Inheritance quiz

What is the *base class* of *Student*?

- **Answer:** Person

Which class is *Student* a *derived class* of?

- **Answer:** Person

What is a *derived class* of *Student*?

- **Answer:** TeachingAssistant

Which data attributes does *TeachingAssistant* inherit?

- **Answer:** first_name, last_name, age, and s_number

Which class is *TeachingAssistant* a *derived class* of?

- **Answer:** Person and Student (sorry, it's both)

How do we implement this in Python?

Remember our beautiful class

```python
class Person():

    # The constructor
    def __init__(self, first_name, last_name, age):
        self.first_name = first_name  # Attribute
        self.last_name = last_name  # Attribute
        self.age = age  # Attribute

    # Method
    def speak(self):
        print(f"Hey, my name is {self.first_name}")
```

Let's derive from it

```
class Student(Person):
```

Let's derive from it

```python
class Student(Person):

    # The constructor
    def __init__(self, first_name, last_name, age, s_number):

        # Calling the constructor of base class
        super().__init__(first_name, last_name, age)
        self.s_number = s_number

    # Method
    def study(self):
        print(f"Student {self.s_number} is studying!")
```

The super() function

super()

The **super()** function refers to the base class of the class. We can use it to access/call methods of the base class.

If we call a method on the super() function, it is just like we would call it on **self**, but than as if it were an object of the base class!

Lets test it

Lets test it

```
student = Student("Daan", "Wichmann", "secret", "secret")
```

Lets test it

```
student = Student("Daan", "Wichmann", "secret", "secret")
student.speak()
>>> Hey, my name is Daan
```

Lets test it

```
student = Student("Daan", "Wichmann", "secret", "secret")
student.speak()
>>> Hey, my name is Daan
student.study()
>>> Student secret is studying!
```

Hierarchies - Another example

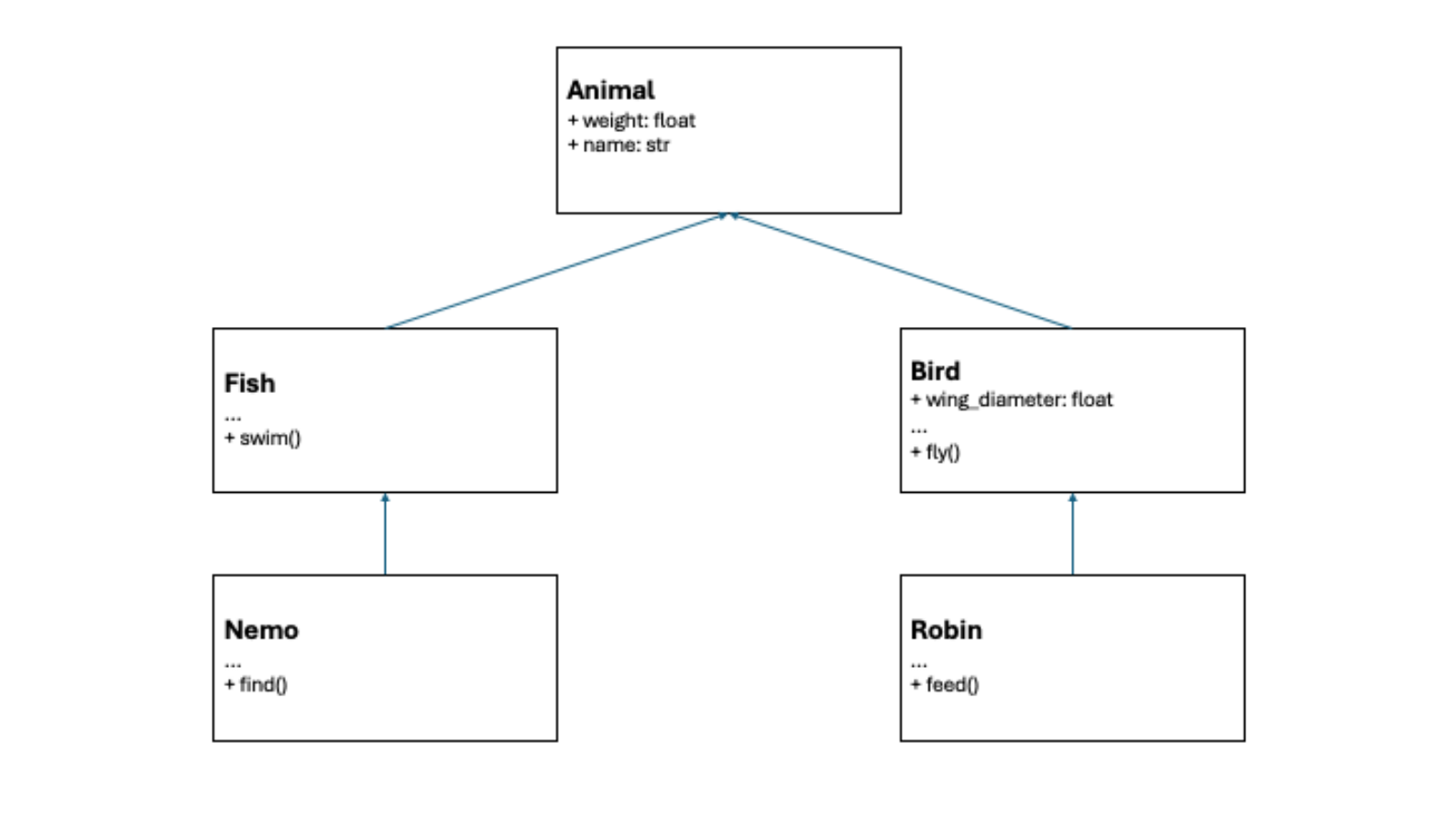

Figure 3: Another example

Method overriding

Overriding

- What if we want Student's *speak()* to do something different from Person's *speak()*?
- We can **override** the method! (**Method overriding**)

Our beautiful class again

```python
class Person():

    # The constructor
    def __init__(self, first_name, last_name, age):
        self.first_name = first_name  # Attribute
        self.last_name = last_name  # Attribute
        self.age = age  # Attribute

    # Method
    def speak(self):
        print(f"Hey, my name is {self.first_name}")
```

And its subclass

```python
class Student(Person):

    # The constructor
    def __init__(self, first_name, last_name, age, s_number):

        # Calling the constructor of base class
        super().__init__(first_name, last_name, age)
        self.s_number = s_number

    # Method
    def study(self):
        print(f"Student {self.s_number} is studying!")
```

Currently

Currently

```
student = Student("Daan", "Wichmann", "secret", "secret")
```

Currently

```
student = Student("Daan", "Wichmann", "secret", "secret")
student.speak()
>>> Hey, my name is Daan
```

Overriding

- We want student to also mention their s-number when speaking!
- **Lets override speak()**

Overriding...

```
class Student(Person):

    # The constructor
    def __init__(self, first_name, last_name, age,
    ↪  s_number):

        # Calling the constructor of base class
        super().__init__(first_name, last_name, age)
        self.s_number = s_number

    # Method
    def study(self):
        print(f"Student {self.s_number} is studying!")

    def speak(self):
        print(f"Hey, my name is {self.first_name} and my
        ↪  s-number is {self.s_number}")
```

And now

And now

```
student = Student("Daan", "Wichmann", "secret", "secret")
```

And now

```
student = Student("Daan", "Wichmann", "secret", "secret")
student.speak()
>>> Hey, my name is Daan and my s-number is secret
```

Operator overloading

Operator overloading

Operator overloading refers to a single operator's capacity to perform several operations based on the class (type) of operands

Now in human terms:

Operator overloading

Now in human terms:

- Operators like addition (+), equals (==), and greater than (>) can perform certain actions based on their type/class.

Operator overloading

Now in human terms:

- Operators like addition (+), equals (==), and greater than (>) can perform certain actions based on their type/class.
- Other operator are:
  - +, -, /, //, %, ==, in, >, <, >=, <=, !=, and more!

Operator overloading

- Let's look at some examples

Operator overloading

- Let's look at some examples
- Addition:

Operator overloading

- Let's look at some examples
- Addition:
  - 5 + 3 = 8 (int)

Operator overloading

- Let's look at some examples
- Addition:
  - 5 + 3 = 8 (int)
  - "Cogn" + "AC" = "CognAC" (str)

Operator overloading

- Let's look at some examples
- Addition:
  - 5 + 3 = 8 (int)
  - "Cogn" + "AC" = "CognAC" (str)
- Greater than:

Operator overloading

- Let's look at some examples
- Addition:
  - 5 + 3 = 8 (int)
  - "Cogn" + "AC" = "CognAC" (str)
- Greater than:
  - 5 > 3 = True (int)

Operator overloading

- Let's look at some examples
- Addition:
  - 5 + 3 = 8 (int)
  - "Cogn" + "AC" = "CognAC" (str)
- Greater than:
  - 5 > 3 = True (int)
  - Person(...) > Person(...) = ???

Operator overloading

- We can define how these operators should behalve in our classes!

Operator overloading

- We can define how these operators should behalve in our classes!
- We use so called double underscore methods (dunder methods, you may forget that name).

What we want

- Consider two objects of our person class: person1 and person2.

What we want

- Consider two objects of our person class: person1 and person2.
- We want person1 > person2 = True, iff person1 is older than person2.

What we want

- Consider two objects of our person class: person1 and person2.
- We want person1 > person2 = True, iff person1 is older than person2.
- Let's **overload** the *greater than* operator in our class Person.

Let's overload

```
class Person():
    ...
```

Let's overload

```
class Person():
    ...
    def __gt__(self, other):  # Note the parameters
```

Let's overload

```
class Person():
    ...
    def __gt__(self, other):  # Note the parameters
        return self.age > other.age
```

Let's test again

Let's test again

```
person1 = Person("Daan", "Wichmann", 19)
person2 = Person("Damai", "Jiworo", 18)
```

Let's test again

```
person1 = Person("Daan", "Wichmann", 19)
person2 = Person("Damai", "Jiworo", 18)
print(person1 > person2)  # What will this return?
```

Let's test again

```
person1 = Person("Daan", "Wichmann", 19)
person2 = Person("Damai", "Jiworo", 18)
print(person1 > person2)  # What will this return?
>>> True
```

Why is 'overloading'

Why is it called, overloading and not overriding?

Why is 'overloading'

Why is it called, overloading and not overriding?

- We are not overriding operators, because they should still behave the same for integers, strings, etc.

Why is 'overloading'

Why is it called, overloading and not overriding?

- We are not overriding operators, because they should still behave the same for integers, strings, etc.
- We are 'loading' the operator with functionality for more classes, i.e. **overloading operators**.

Part 2 of 3 done! Break?

Access Modifiers

Access modifiers

Access modifiers

```
student = Student("Daan", "Wichmann", "secret", "secret")
```

Access modifiers

```
student = Student("Daan", "Wichmann", "secret", "secret")
print(student.s_number)
>>> "secret"
student.s_number = "different"
print(student.s_number)
>>> "different"
```

Access modifiers: the problem

- However, s-numbers generally shouldn't change!
- And if they do, we want to control how they change
- How do we fix this? **Access modifiers**

Access modifiers

Access modifiers control the visibility (access) of the methods and data attributes of a class.

Access modifiers

- Access modifiers specify from where we can access our data attributes/methods, meaning:
  - for **methods**: where we can call them
  - for **data attributes**: where we can access them (get them), and where we can change them (set them)

Access modifiers

- We can modify access to our data attributes and methods at three levels:
  - public
  - protected
  - private
- Data attributes and methods are public by default

The different of access

	Inside the class	In derived classes	Outside the class
public	Yes	Yes	Yes
protected	Yes	Yes	No
private	Yes	No	No

The different of access

	Inside the class	In derived classes	Outside the class
public	Yes	Yes	Yes
protected	Yes	Yes	No
private	Yes	No	No

- Remember this table!!!!

How to modify access???

We can change the access modifier of a data attribute/method by changing the name:

- s_number (public)
- _s_number (protected)
- __s_number (private)

How to modify access???

We can change the access modifier of a data attribute/method by changing the name:

- s_number (public)
- _s_number (protected)
- __s_number (private)

We change the number of underscores before the name!

- public = 0 underscores
- protected = 1 underscore
- private = 2 underscores

Lets update our student class

Modifying the access

```
class Student(Person):
    def __init__(self, first_name, last_name, age, s_number):
        super().__init__(first_name, last_name, age)
```

Modifying the access

```
class Student(Person):
    def __init__(self, first_name, last_name, age, s_number):
        super().__init__(first_name, last_name, age)
        self.__s_number = s_number  # We added two underscores
```

Modifying the access

```
class Student(Person):
    def __init__(self, first_name, last_name, age, s_number):
        super().__init__(first_name, last_name, age)
        self.__s_number = s_number  # We added two underscores

# This part is outside the class
student = Student("Daan", "Wichmann", 21, "s1234567")
print(student.__s_number)
```

- Is s_number a public, protected or private attribute?

Modifying the access

```
class Student(Person):
    def __init__(self, first_name, last_name, age, s_number):
        super().__init__(first_name, last_name, age)
        self.__s_number = s_number  # We added two underscores

# This part is outside the class
student = Student("Daan", "Wichmann", 21, "s1234567")
print(student.__s_number)
```

- Is s_number a public, protected or private attribute?
- Can we print it here?

Modifying the access

```
class Student(Person):
    def __init__(self, first_name, last_name, age, s_number):
        super().__init__(first_name, last_name, age)
        self.__s_number = s_number  # We added two underscores

    def talk(self):
        print(f"Student {self.__s_number} is talking")
```

- Can we access it here?

Remember our hierarchy

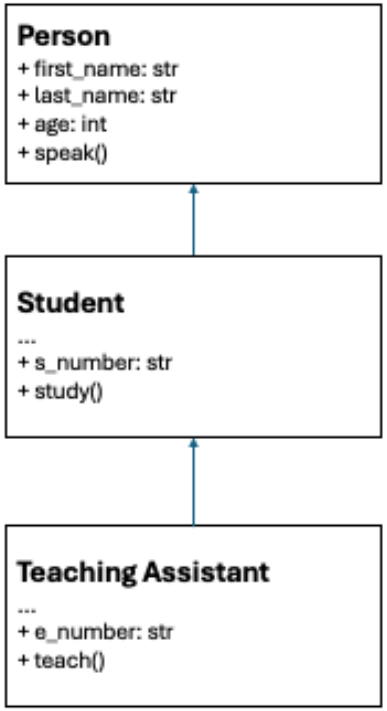

Figure 4: our hierarchy

Modifying the access

```
class Student(Person):
    def __init__(self, first_name, last_name, age,
    ↪  s_number):
        super().__init__(first_name, last_name, age)
        self.__s_number = s_number

# TeachingAssistant derives from Student
class TeachingAssistant(Student)
    def __init__(self, first_name, last_name, age,
    ↪  s_number, e_number):
            # Calling the constructor of Student
            super().__init__(first_name, last_name, age,
            ↪  s_number)
            self.__e_number = e_number

    def talk(self):
        print(f"Student {self.__s_number} is talking")
```

- Can we access it here?

Modifying the access again

```
class Student(Person):
    def __init__(self, first_name, last_name, age,
    ↪  s_number):
        super().__init__(first_name, last_name, age)
        self._s_number = s_number  # Removed one underscore

class TeachingAssistant(Student)
    def __init__(self, first_name, last_name, age,
    ↪  s_number, e_number):
            # Calling the constructor of Student
            super().__init__(first_name, last_name, age,
            ↪  s_number)
            self.__e_number = e_number

    def talk(self):
        print(f"Student {self._s_number} is talking")
```

- Is s_number public, private or protected?

Modifying the access again

```
class Student(Person):
    def __init__(self, first_name, last_name, age,
    ↪  s_number):
        super().__init__(first_name, last_name, age)
        self._s_number = s_number  # Removed one underscore

class TeachingAssistant(Student)
    def __init__(self, first_name, last_name, age,
    ↪  s_number, e_number):
            # Calling the constructor of Student
            super().__init__(first_name, last_name, age,
            ↪  s_number)
            self.__e_number = e_number

    def talk(self):
        print(f"Student {self._s_number} is talking")
```

- Is s_number public, private or protected?
- Can we access it in TeachingAssistant now?

Back to being private

```
class Student(Person):
    def __init__(self, first_name, last_name, age,
    ↪  s_number):
        super().__init__(first_name, last_name, age)
        self.__s_number = s_number  # It's private again!

class TeachingAssistant(Student)
    def __init__(self, first_name, last_name, age,
    ↪  s_number, e_number):
            # Calling the constructor of Student
            super().__init__(first_name, last_name, age,
            ↪  s_number)
            self.__e_number = e_number

    def change_student_number(self, new_student_number):
        self.__s_number = new_student_number
```

- Can we do this?

Back to being private

```
class Student(Person):
    def __init__(self, first_name, last_name, age,
    ↪  s_number):
        super().__init__(first_name, last_name, age)
        self.__s_number = s_number  # It's private again!

class TeachingAssistant(Student)
    def __init__(self, first_name, last_name, age,
    ↪  s_number, e_number):
            # Calling the constructor of Student
            super().__init__(first_name, last_name, age,
            ↪  s_number)
            self.__e_number = e_number

    def change_student_number(self, new_student_number):
        self.__s_number = new_student_number
```

- Can we do this?
- Not being able to access it, means we also shouldn't change it!

Python being Python

Access modifiers in Python (only for technical details)

In Python, access modifiers are not heavily enforced. Meaning, that if we wanted to, we can still access private/protected data attributes in places we shouldn't. However, it is the convention that you shouldn't! (And in the exam it is assumed they are heavily enforced).

- Other languages like Java, do heavily enforce their access modifiers.

Why are we doing this again?

- Why are we making our data attributes and methods private or protected?

Why are we doing this again?

- Why are we making our data attributes and methods private or protected?
- **Encapsulation**

Encapsulation

The problem

Why do we want to make our data attributes protected or private?

The problem

Why do we want to make our data attributes protected or private?

- Sometimes they contain information that shouldn't be known outside of the class

The problem

Why do we want to make our data attributes protected or private?

- Sometimes they contain information that shouldn't be known outside of the class
- **We want to control how they are changed**

The problem

Why do we want to make our data attributes protected or private?

- Sometimes they contain information that shouldn't be known outside of the class
- **We want to control how they are changed**
  - E.g., a student number should always start with a 's' and have 7 numbers.

The problem

Why do we want to make our data attributes protected or private?

- Sometimes they contain information that shouldn't be known outside of the class
- **We want to control how they are changed**
  - E.g., a student number should always start with a 's' and have 7 numbers.
  - A password should contain certain special characters

The problem

Why do we want to make our data attributes protected or private?

- Sometimes they contain information that shouldn't be known outside of the class
- **We want to control how they are changed**
  - E.g., a student number should always start with a 's' and have 7 numbers.
  - A password should contain certain special characters
- Sometimes you want a data attribute to be able to be read, but not be able to be set. And the other way around.

Solution to this problem

Solution: we make methods that change our protected and private data attributes (**setters**) and/or make methods that return the value of our data attribute (**getters**).

Solution to this problem

Solution: we make methods that change our protected and private data attributes (**setters**) and/or make methods that return the value of our data attribute (**getters**).

- We '**encapsulate**' our data attributes with *getters* and/or *setters*.

Solution to this problem

Solution: we make methods that change our protected and private data attributes (**setters**) and/or make methods that return the value of our data attribute (**getters**).

- We '**encapsulate**' our data attributes with *getters* and/or *setters*.
- **This is encapsulation!**

Let’s see how we do this in Python!

Making getters and setters

```
class Student(Person):
    def __init__(self, first_name, last_name, age, s_number):
        super().__init__(first_name, last_name, age)
        self.__s_number = s_number  # Private data attribute
```

Making getters and setters

```
class Student(Person):
    def __init__(self, first_name, last_name, age, s_number):
        super().__init__(first_name, last_name, age)
        self.__s_number = s_number  # Private data attribute

    @property  # This is a decorator that indicates its a getter
```

Making getters and setters

```
class Student(Person):
    def __init__(self, first_name, last_name, age, s_number):
        super().__init__(first_name, last_name, age)
        self.__s_number = s_number  # Private data attribute

    @property  # This is a decorator that indicates its a getter
    def s_number(self):  # Note the name
```

Making getters and setters

```
class Student(Person):
    def __init__(self, first_name, last_name, age, s_number):
        super().__init__(first_name, last_name, age)
        self.__s_number = s_number  # Private data attribute

    @property  # This is a decorator that indicates its a getter
    def s_number(self):  # Note the name
        return self.__s_number
```

Making getters and setters

```
class Student(Person):
    def __init__(self, first_name, last_name, age, s_number):
        super().__init__(first_name, last_name, age)
        self.__s_number = s_number  # Private data attribute

    @property  # This is a decorator that indicates its a getter
    def s_number(self):  # Note the name
        return self.__s_number

    @s_number.setter  # Decorator for setter
```

Making getters and setters

```
class Student(Person):
    def __init__(self, first_name, last_name, age, s_number):
        super().__init__(first_name, last_name, age)
        self.__s_number = s_number  # Private data attribute

    @property  # This is a decorator that indicates its a getter
    def s_number(self):  # Note the name
        return self.__s_number

    @s_number.setter  # Decorator for setter
    def s_number(self, new_s_number):  # Note the name
        if new_s_number.startswith("s"):
            self.__s_number = new_s_number
        else:
            raise ValueError
```

Making getters and setters

```
class Student(Person):
    ...
    @property  # This is a decorator that indicates its a getter
    def s_number(self):
        return self.__s_number

    @s_number.setter  # Decorator for setter
    def s_number(self, new_s_number):
        if new_s_number.startswith("s"):
            self.__s_number = new_s_number
        else:
            raise ValueError

student = Student("Daan", "Wichmann", "Secret", "s1234567")
# Because of the 'property' decorator we can use the getter
# as if s_number is just a normal data attribute
print(student.s_number)
>>> "s1234567"
```

Making getters and setters

```
class Student(Person):
    ...
    @property  # This is a decorator that indicates its a getter
    def s_number(self):
        return self.__s_number

    @s_number.setter  # Decorator for setter
    def s_number(self, new_s_number):
        if new_s_number.startswith("s"):
            self.__s_number = new_s_number
        else:
            raise ValueError

student = Student("Daan", "Wichmann", "Secret", "s1234567")
student.s_number = 1234567
print(student.s_number)
```

What will happen?

Making getters and setters

```
class Student(Person):
    ...
    @property  # This is a decorator that indicates its a getter
    def s_number(self):
        return self.__s_number

    @s_number.setter  # Decorator for setter
    def s_number(self, new_s_number):
        if new_s_number.startswith("s"):
            self.__s_number = new_s_number
        else:
            raise ValueError

student = Student("Daan", "Wichmann", "Secret", "s1234567")
student.s_number = 1234567
print(student.s_number)
```

What will happen?

```
>>> ValueError
```

Making getters and setters

```
class Student(Person):
    ...
    @property  # This is a decorator that indicates its a getter
    def s_number(self):
        return self.__s_number

    @s_number.setter  # Decorator for setter
    def s_number(self, new_s_number):
        if new_s_number.startswith("s"):
            self.__s_number = new_s_number
        else:
            raise ValueError

student = Student("Daan", "Wichmann", "Secret", "s1234567")
student.s_number = s1020425
print(student.s_number)
```

Making getters and setters

```
class Student(Person):
    ...
    @property  # This is a decorator that indicates its a getter
    def s_number(self):
        return self.__s_number

    @s_number.setter  # Decorator for setter
    def s_number(self, new_s_number):
        if new_s_number.startswith("s"):
            self.__s_number = new_s_number
        else:
            raise ValueError

student = Student("Daan", "Wichmann", "Secret", "s1234567")
student.s_number = s1020425
print(student.s_number)
>>> "s1020425"
```

Encapsulation is important

- Encapsulation is one of the most important concepts of OOP!

Encapsulation is important

- Encapsulation is one of the most important concepts of OOP!
- Make sure to study it from the material, before the exam.

Encapsulation is important

- Encapsulation is one of the most important concepts of OOP!
- Make sure to study it from the material, before the exam.

Encapsulation (definition from the material)

Hiding attributes from clients is called encapsulation. As the name implies, the attribute is "enclosed in a capsule". The client is then offered a suitable interface for accessing and processing the data stored in the object.

Encapsulation is important

- Encapsulation is one of the most important concepts of OOP!
- Make sure to study it from the material, before the exam.

Encapsulation (definition from the material)

Hiding attributes from clients is called encapsulation. As the name implies, the attribute is "enclosed in a capsule". The client is then offered a suitable interface for accessing and processing the data stored in the object.

- Client := your program
- The suitable interface that is referred to are the getters and setters

Seperation of Concerns

Separation of Concerns

Separation of concerns is a design principle for separating a computer program into distinct sections such that each section addresses a separate concern. A concern is a set of information that affects the code of a computer program.

Separation of Concerns

- One of the most important concepts of OOP! (together with encapsulation).
- Classes separate our program into different parts
- This makes it so that one part of the program does not have to be **concerned** with another part of our program.
- **Encapsulation** facilitates this by hiding the implementation of our classes behind well defined interfaces (e.g. getters and setters).

Separation of Concerns

- One of the most important concepts of OOP! (together with encapsulation).
- Classes separate our program into different parts
- This makes it so that one part of the program does not have to be **concerned** with another part of our program.
- **Encapsulation** facilitates this by hiding the implementation of our classes behind well defined interfaces (e.g. getters and setters).
  - So other classes don't have to be concerned with how a variable is changed or read.
- **In a nutshell:** a class should only do what it is designed to do.

Why you should use OOP!

Why you should know about seperation of concerns and encapsulation.

Why you should use OOP!

Let's say you have a group project, to create a phone book app.

Why you should use OOP!

Let's say you have a group project, to create a phone book app.

- You create a **PhoneBook** class that stores phone numbers.

Why you should use OOP!

Let's say you have a group project, to create a phone book app.

- You create a **PhoneBook** class that stores phone numbers.
- You want to be able to add and see phone numbers via the terminal.

Why you should use OOP!

Let's say you have a group project, to create a phone book app.

- You create a **PhoneBook** class that stores phone numbers.
- You want to be able to add and see phone numbers via the terminal.
- (Following seperation of concerns) you create a different class **PhoneBookApplication** that handles the input, output, and the loop of your program.

Why you should use OOP!

Let's say you have a group project, to create a phone book app.

- You create a **PhoneBook** class that stores phone numbers.
- You want to be able to add and see phone numbers via the terminal.
- (Following seperation of concerns) you create a different class **PhoneBookApplication** that handles the input, output, and the loop of your program.
- You also want to maintain a database with all phone numbers (e.g. txt file, CSV file, or MySQL database).

Why you should use OOP!

Let's say you have a group project, to create a phone book app.

- You create a **PhoneBook** class that stores phone numbers.
- You want to be able to add and see phone numbers via the terminal.
- (Following seperation of concerns) you create a different class **PhoneBookApplication** that handles the input, output, and the loop of your program.
- You also want to maintain a database with all phone numbers (e.g. txt file, CSV file, or MySQL database).
- So, you create another class that interacts with a database **DatabaseHandler**

Why you should use OOP!

- Let's assume person A made the PhoneBook class, person B made the PhoneBookApplication class, and person C made the DatabaseHandler class.

Why you should use OOP!

- Let's assume person A made the PhoneBook class, person B made the PhoneBookApplication class, and person C made the DatabaseHandler class.
- By creating different classes you can divide the work nicely!

Why you should use OOP!

- Let's assume person A made the PhoneBook class, person B made the PhoneBookApplication class, and person C made the DatabaseHandler class.
- By creating different classes you can divide the work nicely!
- Lets assume all classes have well defined methods.

Why you should use OOP!

- Let's assume person A made the PhoneBook class, person B made the PhoneBookApplication class, and person C made the DatabaseHandler class.
- By creating different classes you can divide the work nicely!
- Lets assume all classes have well defined methods.
- For example, the DatabaseHandler has a method save_to_database(phonebook).

Why you should use OOP!

- Let's assume person A made the PhoneBook class, person B made the PhoneBookApplication class, and person C made the DatabaseHandler class.
- By creating different classes you can divide the work nicely!
- Lets assume all classes have well defined methods.
- For example, the DatabaseHandler has a method save_to_database(phonebook).
- Only person C has to be concerned with how this method works, and has to make sure that it saves to a database.

Why you should use OOP!

- Let's assume person A made the PhoneBook class, person B made the PhoneBookApplication class, and person C made the DatabaseHandler class.
- By creating different classes you can divide the work nicely!
- Lets assume all classes have well defined methods.
- For example, the DatabaseHandler has a method save_to_database(phonebook).
- Only person C has to be concerned with how this method works, and has to make sure that it saves to a database.
- Person B needs to use this method, but does not need to be **concerned** with how it works! The implementation is **encapsulated** behind the method.

Why you should use OOP!

- Separation of Concerns and encapsulation makes creating big projects (in groups) way easier!

Class attributes

What are class attributes?

What are class attributes?

Class attributes (definition)

Class attributes are **data attributes** or **methods**, that do not belong to instances of a class, but to the class itself.

What are class attributes?

Class attributes (definition)

Class attributes are **data attributes** or **methods**, that do not belong to instances of a class, but to the class itself.

- Class data attributes

What are class attributes?

Class attributes (definition)

Class attributes are **data attributes** or **methods**, that do not belong to instances of a class, but to the class itself.

- Class data attributes
- Class methods

An example...

An example...

```
class Person():
    ...

    # An oridinary method :o
    def greet(self):
        print("Hey!")
```

Hmmmmm...

Hmmmmm...

```
person1 = Person("Daan", "Wichmann", 19)
```

Hmmmmm...

```
person1 = Person("Daan", "Wichmann", 19)
person2 = Person("Ella", "Kemperman", 19)
```

Hmmmmm...

```
person1 = Person("Daan", "Wichmann", 19)
person2 = Person("Ella", "Kemperman", 19)
person1.greet()
>>> "Hey!"
```

Hmmmmm...

```
person1 = Person("Daan", "Wichmann", 19)
person2 = Person("Ella", "Kemperman", 19)
person1.greet()
>>> "Hey!"
person2.greet()
>>> "Hey!"
```

Hmmmmm...

```
person1 = Person("Daan", "Wichmann", 19)
person2 = Person("Ella", "Kemperman", 19)
person1.greet()
>>> "Hey!"
person2.greet()
>>> "Hey!"
```

- Different objects, same output!

Hmmmmm...

```
person1 = Person("Daan", "Wichmann", 19)
person2 = Person("Ella", "Kemperman", 19)
person1.greet()
>>> "Hey!"
person2.greet()
>>> "Hey!"
```

- Different objects, same output!
- We can do this better!

Wow it's a classmethod!

Wow it's a classmethod!

```
class Person():
    ...

    @classmethod # Only the decorator is added!
    def greet(cls):  # Note the first parameter!
        print("Hey!")
```

Same for data attributes!

Same for data attributes!

- If the values are the same for every object, we can make it a class data attribute!

```
class Person():
    greeting = "Hey! # Class data attribute

    @classmethod
    def greet(cls):
        print(cls.greeting)
```

That is it!
(make sure to still look at the material!)

NB

I did not cover **class attributes**. Please study this in your own time, because it is still fairly important! (Part 9.5 of the material)

Extra info

If you are interested in programming, I highly recommend following: Object Orientend Programming (NWI-IPI005)

You can find these slides and the code from the presentation on my github: `https://github.com/D21W12`